

COL 774: Machine Learning

Assignment 4

Learning to Solve Mazes: Sequence-to-Sequence Models

Ashvath Hari Bhaskar

Jayesh Narayanan

Entry Number: 2023EE10168

Entry Number: 2023EE11048

November 2025

Contents

1	RNN with Bahdanau Attention	2
1.1	Model Architecture	2
1.1.1	Encoder	2
1.1.2	Bahdanau Attention Mechanism	2
1.1.3	Decoder	3
1.2	Hyperparameters	3
1.3	Implementation Details	4
1.3.1	Training vs. Evaluation Strategy	4
1.3.2	Data Handling	4
1.4	Training Results	4
1.4.1	Performance Analysis	5
1.5	Learning Curves	5
1.6	Visualization of Predictions	6
1.7	Analysis and Observations	6
2	Transformer Architecture	8
2.1	Model Architecture Overview	8
2.1.1	Input Representation and Embedding	8
2.1.2	Positional Encoding	8
2.1.3	Data Processing Pipeline	8
2.1.4	Hyperparameters	10
2.1.5	Training Details	10
2.2	Results	10
2.2.1	Performance Metrics	10
2.2.2	Training Curves	10
2.2.3	Sample Predictions	11
2.3	Conclusions	11
2.3.1	Model is Learning Effectively but Has Room for Improvement	11
2.3.2	Slight Overfitting is Present	12
2.3.3	Good Generalization Between Validation and Test Sets	12
2.3.4	Loss Convergence Suggests Near-Optimal Learning Rate	12

1 RNN with Bahdanau Attention

1.1 Model Architecture

We implemented a Sequence-to-Sequence (Seq2Seq) model comprised of an Encoder and a Decoder, enhanced with a Bahdanau (Additive) Attention mechanism to improve the model’s ability to capture long-range dependencies in maze structures.

1.1.1 Encoder

The Encoder is responsible for reading the variable-length input sequence representing the maze structure (adjacency list, origin, and target) and transforming it into a continuous latent representation. It is implemented as a subclass of `nn.Module` with the following components:

- **Embedding Layer:** An `nn.Embedding` layer maps discrete token indices to 128-dimensional dense vectors. It utilizes `padding_idx=0` to ensure that padding tokens do not contribute to the gradient or learned representation.
- **Sequence Packing:** To efficiently process batches containing sequences of varying lengths, the input embeddings are compressed using `pack_padded_sequence` before being passed to the RNN. This optimization allows the model to ignore padding tokens during the recurrent computation.
- **Recurrent Architecture:** The core of the encoder is a 2-layer vanilla `nn.RNN` with a hidden dimension of 512. It operates with `batch_first=True`, processing the packed sequence to capture structural dependencies within the maze data.
- **Outputs:** The encoder produces two key outputs:
 1. A sequence of hidden states for every time step (unpacked via `pad_packed_sequence`), which serves as the memory bank for the attention mechanism.
 2. The final hidden state of the last timestep, which is used to initialize the hidden state of the Decoder.

1.1.2 Bahdanau Attention Mechanism

To allow the decoder to focus on specific parts of the maze structure at each step of the path generation, we implemented an Additive (Bahdanau) Attention mechanism. This module calculates a dynamic context vector based on the alignment between the current decoder state and the entire history of encoder states. The mechanism consists of three learnable linear layers: W_1 and W_2 project the decoder hidden state and encoder outputs into a shared latent space, while a third layer V collapses the representation into a scalar alignment score. The process for each decoding step i proceeds as follows:

1. **Alignment Score (Energy):** We calculate the energy e_{ij} for each encoder hidden state h_j using a feed-forward neural network with a $\tanh$ activation:

$$e_{ij} = V^T \tanh(W_1 s_{i-1} + W_2 h_j) \quad (1)$$

2. **Masking:** To prevent the model from attending to padding tokens (which were added during batch collation), we apply a mask to the energy scores. Scores corresponding to padding indices are set to negative infinity ($-\infty$), ensuring their contribution becomes zero after the softmax operation.

3. **Attention Weights:** The masked energy scores are normalized using the softmax function to produce a probability distribution α_{ij} over the input sequence:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})} \quad (2)$$

4. **Context Vector:** Finally, the context vector c_i is computed as the weighted sum of the encoder hidden states:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j \quad (3)$$

1.1.3 Decoder

The Decoder is designed as an auto-regressive recurrent network that constructs the solution path step-by-step. It learns to predict the next coordinate y_t given the previous coordinate y_{t-1} and the context vector c_t derived from the attention mechanism.

- **Input Combination:** At each time step t , the previous token y_{t-1} is first passed through an embedding layer (dimension 128) and a dropout layer. This embedded vector is then concatenated with the dynamic context vector c_t (dimension 512) to form the input for the RNN cell.
- **RNN Architecture:** The core decoder uses a 2-layer vanilla `nn.RNN` with a hidden state size of 512. It is initialized with the final hidden state of the Encoder to provide a global context of the maze before decoding begins.
- **Output Projection:** The output of the RNN at each step is passed through a fully connected linear layer (`self.fc`) which projects the 512-dimensional hidden state back to the vocabulary size (50 tokens), producing the logits for the next token prediction.
- **Decoding Strategy:**
 - **Training:** We employ *Teacher Forcing* with a probability of 0.5, where the ground truth token is fed as the next input to stabilize training.
 - **Inference:** We use greedy decoding, where the token with the highest probability is selected and fed back as the input for the next step, continuing until the `<PATH_END>` token is generated or the maximum length is reached.

1.2 Hyperparameters

Table 1: RNN Hyperparameters (Matched to Code)

Parameter	Value
Batch Size	32
Epochs	20
Learning Rate	1e-4 (0.0001)
Embedding Dimension	128
Hidden Dimension	512
Number of RNN Layers	2
Dropout	0.0
Optimizer	Adam
Teacher Forcing Ratio	0.5

1.3 Implementation Details

1.3.1 Training vs. Evaluation Strategy

A critical distinction in our implementation is the use of distinct decoding strategies for training versus evaluation to ensure rigorous performance assessment.

- **Training (Teacher Forcing):** During the training phase, we employ a Teacher Forcing ratio of 0.5. At each decoding step, the model has a 50% probability of receiving the ground truth token from the previous step as input, rather than its own generated prediction. This technique stabilizes the gradients early in training and accelerates convergence.
- **Evaluation (Autoregressive "True Metrics"):** For both validation and testing, we strictly disable teacher forcing (ratio = 0.0). We implemented a custom evaluation loop that generates sequences **autoregressively**: the model predicts a token, and that predicted token is fed back as the input for the next time step. This simulates real-world inference and ensures that reported accuracies reflect the model’s ability to recover from its own errors without external guidance.
- **Metrics:** We report three key metrics:
 1. **Loss:** Standard Cross-Entropy Loss (ignoring padding indices).
 2. **Token Accuracy:** The percentage of correctly predicted coordinate tokens generated autoregressively.
 3. **Sequence Accuracy (Exact Match):** The percentage of mazes where the *entire* predicted path exactly matches the ground truth path from start to finish.

1.3.2 Data Handling

- **Dataset Splitting:** The provided training data was split into a training set of 64,000 samples (80%) and a held-out validation set of 16,000 samples (20%). The separate test set contains 20,000 samples used solely for final evaluation.
- **Vocabulary:** We constructed a vocabulary of 50 unique tokens, which includes all coordinate pairs (e.g., (0,0), (5,5)), special navigation tokens (<PATH_START>, <PATH_END>), and padding/unknown tokens (<PAD>, <UNK>).
- **Sequence Processing:** Input sequences (maze layouts) and target sequences (solution paths) are tokenized and converted to integer indices. The target sequences are shifted to create separate `decoder_input` (without <PATH_END>) and `decoder_target` (without <PATH_START>) tensors to facilitate the teacher forcing mechanism described above.

1.4 Training Results

The model was trained for 20 epochs using a hybrid strategy: teacher forcing was applied during the training step to stabilize learning, while validation metrics were computed autoregressively to reflect true generalization performance. The best model checkpoint, determined by the lowest validation loss (1.8724), was saved at Epoch 13. Although the training loss continued to decrease until Epoch 20 (reaching 0.1357), the validation loss began to diverge slightly, indicating the onset of overfitting.

Table 2: RNN Training Results Summary (Final Evaluation)

Metric	Train (Final)	Validation (Final)	Test Set
Sequence Accuracy (Exact Match)	79.07%	57.83%	58.73%
Token Accuracy	75.75%	49.15%	-
Token F1-Score (LCS Order-Aware)	-	-	0.7903
Loss	0.1357	2.0308	-

1.4.1 Performance Analysis

Test Set Performance: The model achieved an exact sequence match accuracy of **58.73%** on the held-out test set. This means that for nearly 60% of the unseen mazes, the model generated the entire optimal path correctly from start to finish without a single error.

The "Token vs. Sequence" Anomaly: A non-intuitive observation from our results is that the *Sequence Accuracy* (57.83% Val) is consistently higher than the *Token Accuracy* (49.15% Val). Typically, one expects token accuracy to be higher, as a model can predict many correct tokens even in an incorrect path. This inversion is due to our strict evaluation metric which heavily penalizes length mismatches. When the model fails to solve a maze, it often generates paths that differ significantly in length from the ground truth (e.g., getting stuck in a loop or terminating early). These length deviations add a large penalty to the token accuracy denominator, dragging the average score down significantly. In contrast, the Sequence Accuracy is a binary metric; it captures the high proportion of perfect successes without being diluted by the magnitude of the failures.

F1-Score (Partial Matching): To better assess partial correctness for mazes that were not solved perfectly, we calculated an F1-score of **0.7903** using Longest Common Subsequence (LCS) logic. This high score indicates that even when the model fails to produce an exact match, it correctly identifies a significant portion of the valid path segments and maintains the correct relative ordering of coordinates.

1.5 Learning Curves

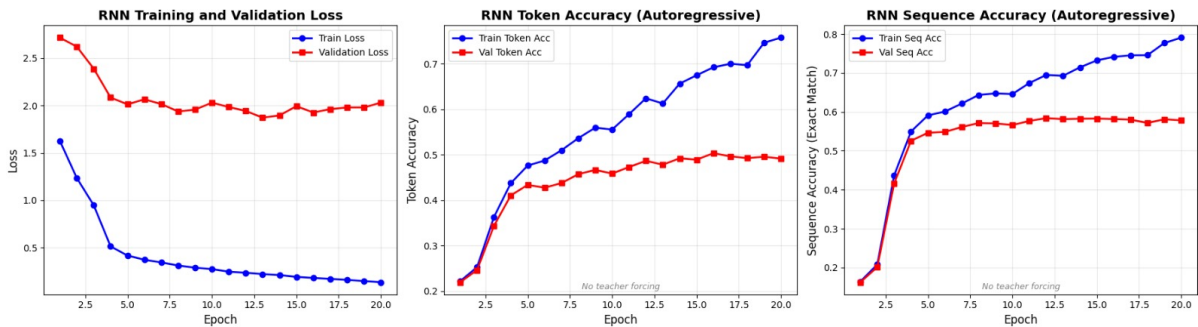


Figure 1: RNN Training Dynamics across 20 epochs

Training loss decreases significantly from 1.62 to 0.14, while validation loss reaches its minimum of 1.87 at Epoch 13 before diverging slightly to 2.03. Middle: Token accuracy reaches 75.8% (train) and 49.2% (val), reflecting the difficulty of maintaining perfect alignment autoregressively. Right: Sequence accuracy demonstrates rapid learning in the first 8 epochs (improving from 16.2% to 57.1% on validation), peaking at 58.4% (Epoch 12). The final evaluation on the test set yields an Exact Sequence Match of 58.73% and a Token F1-score of 0.79.

1.6 Visualization of Predictions

Below are 5 randomly selected mazes from the validation set showing ground-truth paths (green) and RNN predictions (red/dashed). These examples illustrate the model’s performance on different maze configurations, including both successful exact matches and cases where the model produces partially correct paths.

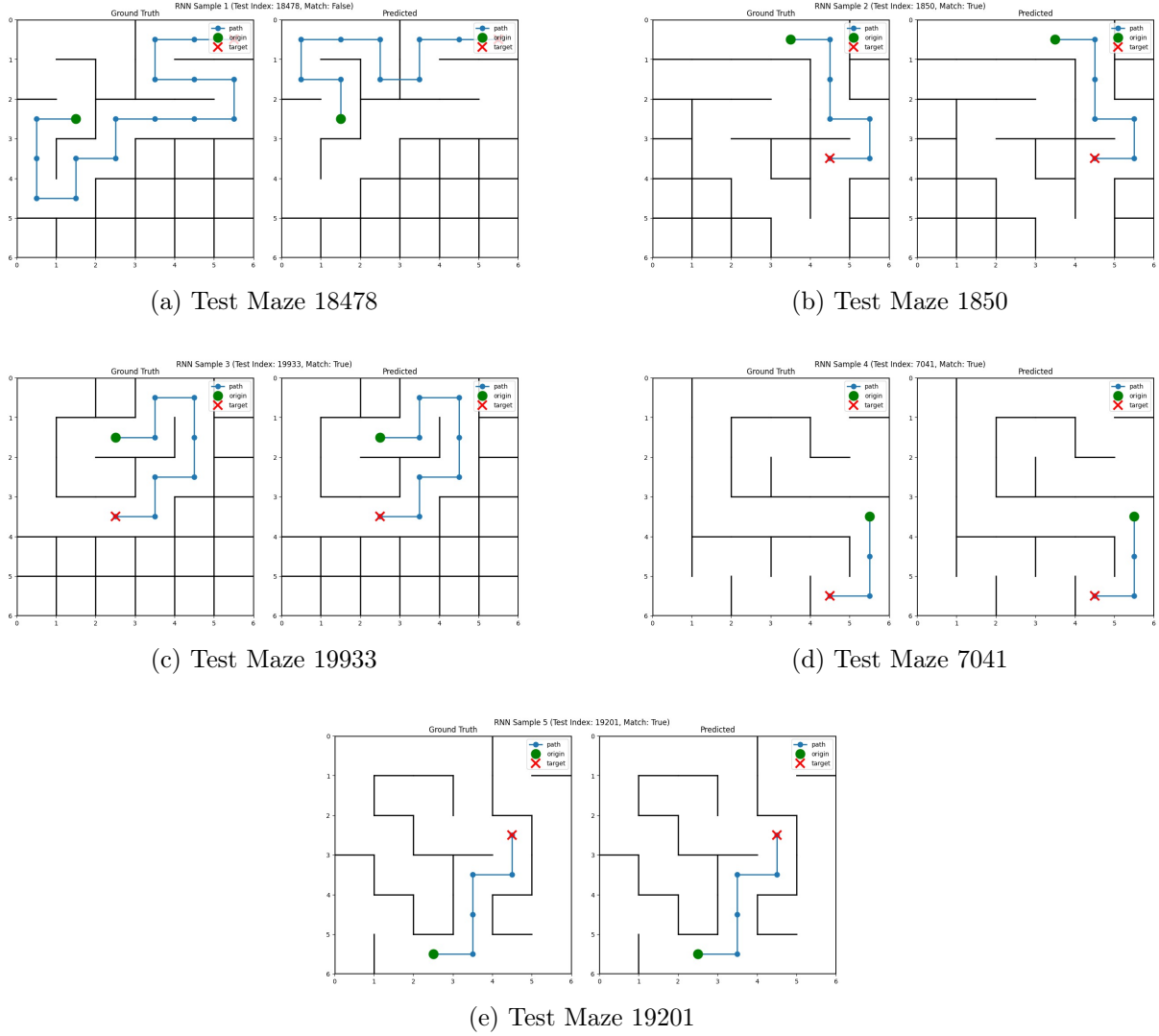


Figure 2: RNN Predictions on Validation Set. Green: Ground Truth, Red (dashed): Prediction

1.7 Analysis and Observations

1. **Convergence Behavior:** The model demonstrated rapid feature learning in the initial phase. By Epoch 4, the Sequence Accuracy had already jumped to approximately 55% (Training), indicating the model quickly grasped the fundamental syntax of coordinate sequences and basic adjacency rules. However, performance gains slowed significantly after Epoch 10.
2. **Loss Dynamics:** Training loss decreased consistently from 1.62 to 0.14 across 20 epochs. However, the validation loss reached a minimum of 1.87 at Epoch 13 before diverging to 2.03. This growing gap between training and validation loss indicates clear overfitting in the later stages, likely due to the absence of dropout.

3. **Token vs Sequence Accuracy Anomaly:** A significant and non-intuitive gap exists where Sequence Accuracy (57.83% Val) is *higher* than Token Accuracy (49.15% Val). In standard text generation, token accuracy is usually higher. This inversion suggests that when the model fails, it fails catastrophically (e.g., generating a path of the wrong length), which penalizes the token metric heavily. Conversely, it achieves perfect exact matches frequently without accumulating partial credit errors.
4. **Generalization Performance:** The model shows a notable generalization gap. While it achieved 79.07% sequence accuracy on the training set (with teacher forcing help), it capped at 58.73% on the unseen test set (autoregressive). This indicates that while it memorized training paths well, it struggled to robustly apply navigation rules to novel maze configurations under autoregressive conditions.
5. **Exact Match vs. Partial Success (F1):** Despite the moderate Exact Match score (58.73%), the LCS-based F1 score is significantly higher at 0.7903. This confirms that the model successfully captures the general direction and structure of the solution in the vast majority of cases, even if it fails to produce a perfect, error-free path.
6. **Teacher Forcing & Exposure Bias:** The large disparity between final Training Token Accuracy (75.75%) and Validation Token Accuracy (49.15%) highlights the impact of “Exposure Bias.” The model performs well when corrected at every step (training) but struggles to recover from its own cascading errors during independent inference (validation/testing).
7. **Training Efficiency:** The training was computationally efficient, with each epoch taking approximately 3 minutes and 20 seconds on a Tesla T4 GPU. The model achieved near-peak validation performance by Epoch 13, suggesting that shorter training runs with stronger regularization might yield better results than prolonged training.

2 Transformer Architecture

2.1 Model Architecture Overview

The proposed solution utilizes a Sequence-to-Sequence (Seq2Seq) Transformer architecture. The model, defined as **TransformerMazeSolver**, consists of an encoder-decoder structure designed to map a discrete input sequence (maze configuration) to a discrete output sequence (solution path). The architecture fundamentally relies on the self-attention mechanism to capture long-range dependencies within the maze structure without the recurrence limitations and captures the position of each token using positional encoding.

2.1.1 Input Representation and Embedding

The raw input consists of tokenized integer sequences representing the maze state. These integers are mapped to continuous vector representations using a learnable higher dimensional embedding layer, $E \in R^{V \times d_{model}}$, where V is the vocabulary size and d_{model} is the dimensionality of the model (set to 128). This embedding layer is shared between the encoder and decoder. To maintain variance stability during training, the embedding weights are scaled by $\sqrt{d_{model}}$ prior to entering the encoder and decoder stacks.

2.1.2 Positional Encoding

Unlike Recurrent Neural Networks, the standard Transformer architecture is permutation-invariant; the self-attention mechanism computes relationships between tokens based solely on their content, not their positions. To preserve the sequential order of the maze path and structural layout, we inject explicit positional information into the input embeddings.

We employ a fixed sinusoidal positional encoding rather than learnable positional embeddings. This choice is motivated by two key factors:

1. **Generalization to Sequence Lengths:** Sinusoidal functions allow the model to extrapolate to sequence lengths longer than those encountered during training, as the deterministic function is defined for any position pos .
2. **Relative Positioning:** The sinusoidal formulation allows the model to easily learn to attend by relative positions, as strictly defined linear transformations can express PE_{pos+k} as a function of PE_{pos} .

Mathematically, for a position pos and dimension i , the encoding is defined as:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (4)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (5)$$

This vector is added element-wise to the scaled token embeddings before entering the transformer blocks.

2.1.3 Data Processing Pipeline

The data flow through the **TransformerMazeSolver** proceeds through the following computational stages:

Mask Generation

To ensure valid attention computation, two types of boolean masks are generated:

- **Padding Masks:** Applied to both source (*src*) and target (*tgt*) sequences to prevent the model from attending to padding tokens (index 0) used for batch alignment.
- **Causal Look-ahead Mask:** Applied strictly to the decoder. This is a strictly upper-triangular matrix with values of $-\infty$. It ensures that when predicting the token at position t , the decoder can only attend to positions 0 through t , preventing information leakage from future tokens (teacher forcing).

Encoder Forward Pass

The source sequence (maze description) serves as the input to the Encoder. The pipeline proceeds as follows:

1. The source indices are embedded and summed with the positional encodings.
2. The tensor passes through $N = 6$ encoder layers. Each layer applies multi-head self-attention followed by a position-wise feed-forward network.
3. The output is a memory tensor $M \in R^{B \times L_{src} \times d_{model}}$ (variable `memory`), representing the contextualized encoding of the entire maze structure.

Decoder Forward Pass

The decoder processes the target sequence (the solution path shifted right) alongside the encoder output M :

1. The target sequence is embedded and positionally encoded.
2. The tensor passes through $N = 6$ decoder layers. Each layer performs two attention operations:
 - **Masked Self-Attention:** Attends to the target sequence itself, constrained by the Causal Look-ahead Mask.
 - **Cross-Attention:** Queries information from the Encoder’s output memory M using the decoder’s current state.

Output Projection and Inference

The final decoder output is projected via a linear transformation (`fc_out`) to the vocabulary size V , producing logits. During training, these logits are compared against ground truth labels.

During inference (prediction), the model operates autoregressively: the encoder processes the source once to generate M . The decoder is initialized with a *Start of Sequence* token. At each step t , the model predicts the next token based on the distribution of logits at position t . This predicted token is concatenated to the input for step $t + 1$, continuing until an *End of Sequence* token is generated.

2.1.4 Hyperparameters

Table 3: Transformer Hyperparameters

Parameter	Value
Batch Size	32
Epochs	20
Learning Rate	1e-4
d_model	128
Number of Attention Heads	8
Encoder/Decoder Layers	6
Feed-forward Dimension	512
Dropout	0.1
Optimizer	Adam

2.1.5 Training Details

- **Loss Function:** CrossEntropyLoss with `ignore_index=0` (padding)
- **Gradient Clipping:** Max norm 1.0
- **Data Split:** 80% train, 20% validation
- **Masking:** Causal mask for decoder self-attention, padding masks for both encoder and decoder

2.2 Results

2.2.1 Performance Metrics

Table 4: Transformer Test Results

Metric	Test Set
Sequence Accuracy	37.41%
Token Accuracy	32.54%
Token F1-Score	66.65
Loss	0.9265

2.2.2 Training Curves

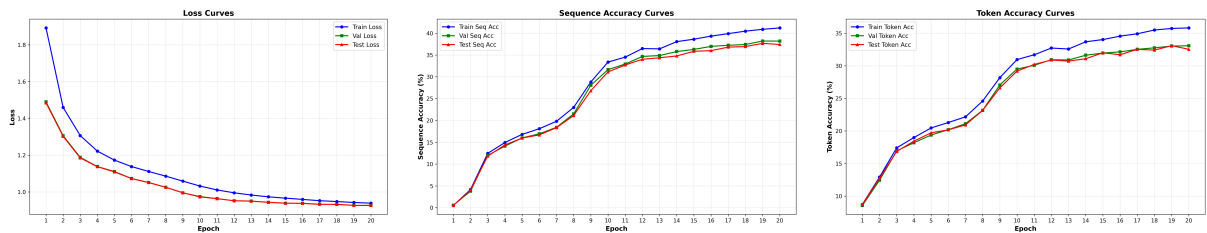
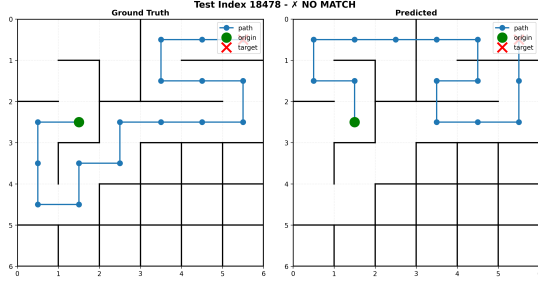


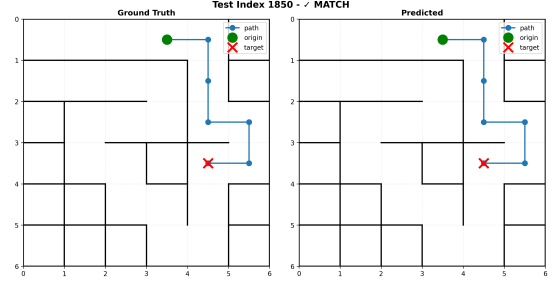
Figure 3: Training dynamics across epochs. Left: Model Loss. Middle: Sequence-level Accuracy. Right: Token-level Accuracy. Curves show performance on train, validation, and test sets.

2.2.3 Sample Predictions

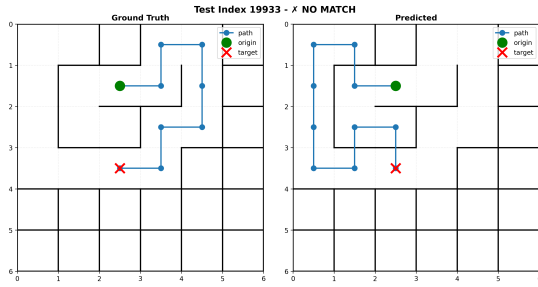
Below are 5 sample predictions from the test set. Green shows ground truth paths, red (dashed) shows model predictions.



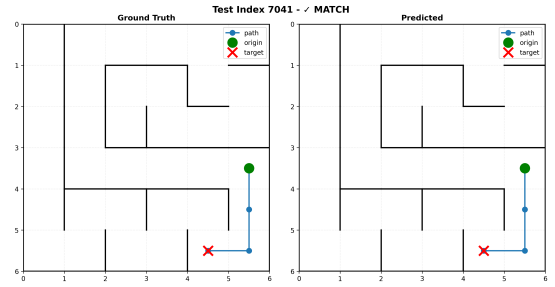
(a) Test Sample 1



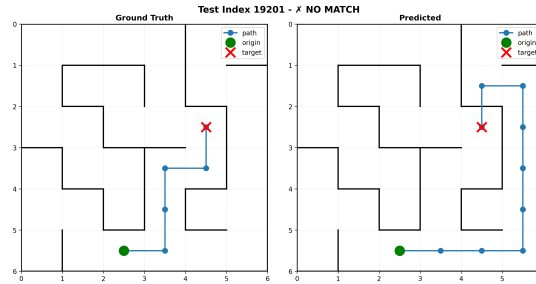
(b) Test Sample 2



(c) Test Sample 3



(d) Test Sample 4



(e) Test Sample 5

Figure 4: Transformer predictions on test set

2.3 Conclusions

Based on the training graphs showing loss and accuracy over 20 epochs, the following conclusions can be drawn:

2.3.1 Model is Learning Effectively but Has Room for Improvement

The sequence accuracy reaches approximately 41% on training data and 37-38% on validation/test sets by epoch 20. The steady improvement throughout all 20 epochs, particularly the consistent rise in training accuracy, indicates the model is still learning and hasn't plateaued. Training for additional epochs could likely yield better performance.

2.3.2 Slight Overfitting is Present

There is a consistent gap between training accuracy ($\sim 41\%$) and validation/test accuracy ($\sim 37\text{--}38\%$) that widens as training progresses. Similarly, the training loss (~ 0.93) remains lower than test loss (~ 0.93) by the end of training. This gap of 3-4% suggests mild overfitting—the model is memorizing some training patterns rather than fully generalizing. However, the overfitting is relatively controlled and not severe.

2.3.3 Good Generalization Between Validation and Test Sets

The validation and test curves track each other very closely throughout training for both loss and accuracy, which is an excellent sign. This indicates the data split is representative and the model's performance on the validation set is a reliable predictor of test performance. There is no unexpected degradation on the test set.

2.3.4 Loss Convergence Suggests Near-Optimal Learning Rate

The loss curves show smooth, steady decline without oscillations or sudden jumps. All three curves (train/val/test) converge toward similar values ($\sim 0.93\text{--}0.95$), suggesting the learning rate and optimization settings are well-tuned. The fact that losses are still slowly decreasing at epoch 20 indicates the model could benefit from additional training epochs or a learning rate schedule with a decay phase.